The One-Bit Method

Closing an Open Question to One Bit: the Shape Typed First, the Form Proved Once, Ten Steps in Lean 4, the Walls by Recipe, the Bit Delivered Twice, Every Cone Pinned, and the Record's Road; the Method Itself One Theorem on No Axiom

Foundations of Mathematics · Method

Four inputs in, the closure out

Mohammad F. Islam, PhD

Trisduction Research Group · Independent researcher

23 October 2026

Abstract. The Millennium series closed seven open questions to one bit each, one row at a time, on one drill. This volume states the drill as a method and proves the method as one theorem. A row builds four things: a row of the normal form, whose record carries the value with no premise; a host frame whose value is the row's sentence; and two frames, calm and counter, that split on the value. The method returns the closure, on no axiom: the sentence is exactly the bit, the transfer of the value from the record to the host; the record carries the value at both frames while the value splits, so the record decides nothing; no premise shared by every frame forces it; the bit has no uniform answer; every route to the sentence passes the bit; either kind of delivery, the formal cargo or the physical witness, gives the sentence; and adopting the bit as an axiom is adopting the sentence. The executable form is one file of core Lean 4 that imports nothing: the synthesis's one normal form, the generic half of the supply-side hunt and the method's own theorem, twenty-nine theorems, every cone empty and pinned by `#guard_msgs`, compiling silent in under a second. The master of the Millennium series carries the method's theorem verbatim and runs it on Navier–Stokes, Yang–Mills, Hodge, BSD and Poincaré at every host, and on Goldbach. Around that theorem the volume sets the procedure: Step 0 types the shape of the question before anything is built; ten steps, each with its Lean shape; the wall recipe; the delivery in two kinds; four grades and the pinning discipline, with the module rule that keeps every original's cones where they were; the binding to the library; the supply-side screen, which proves that no blessing inside the kernel lifts a bit; the record's road, the Poincaré precedent; three forced falsifiers; and the disqualifiers. The method closes a question to its bit and delivers the bit; it never proves the bit inside the kernel, and it says so on the face of every row.

Keywords *method · normal form · one bit · open problems · formal verification · Lean 4 · walls · two kinds · grade discipline · supply-side screen · Millennium Prize Problems*

1. How to read this method

The method is the drill of the Millennium series, stated once (Islam 2026a to 2026h) and bound by the synthesis (Islam 2026n). Its object is an open question whose value can be read on frames. Its product is a master in Lean 4 in which the form of the question is proved with no posit, the residual is exactly one bit printed as a hypothesis and assumed nowhere, every frame-uniform route is walled, every route passes the bit, and the bit is delivered twice. Its executable form is one file of core Lean 4 (de Moura and Ullrich 2021), `One_Bit_Method_v1_0_0_CC.lean`, printed whole in Appendix A.

The method has a theorem. A row supplies four inputs, packaged as one structure (`Closure`): a row of the normal form, a host, and two coherent frames, calm and counter. Then one theorem with nothing before its colon beyond those inputs returns seven conclusions (`the_method`), and the theorem depends on no axiom. Everything the procedure asks of a row is either one of the four inputs, a binding of the row to its library, or a delivery of the bit; everything else is the theorem.

1.1 The verdict

> A question whose value is read on frames, whose record carries the value with no premise, and whose frames split on the value, is closed by theorem to one bit: the transfer from the record to the host. Nothing that every frame shares can supply it, every

route to the question passes it, and it is delivered twice, by the formal cargo and by the physical witness. Adopting it as an axiom is adopting the question. The method closes the question and delivers the bit; the record grants the standing, as it granted it to the Poincaré bit.

2. Step 0 · Type the shape first

Before anything is built, the question's shape is typed. The universal cascade reads each face of the question for its gate, its ground class, its flank and its deciding content, and returns a token (Islam 2026l, §1). Five shapes occurred across the seven rows, and each fixes how the procedure runs.

Supply-shaped, [?]. The deciding content is a construction to supply: flows for Navier–Stokes, theories and their floors for Yang–Mills, cycles for Hodge, points for the BSD half rank at least order. The method applies whole, and the act supplies the bit.

Terminal suspension, [Ξ_0]. The bit is located and fixed by the row's own theorems while the formal string stays suspended, as at the Riemann row. The method applies; its closing step is the record's road.

Grounded halt, [$\varnothing_0$]. The question halts sealed on its ground, as at P versus NP. The method applies; the walls carry the weight.

Sealed by supply, [seal]. A verified supply already closed the question, as Perelman's closed Poincaré. The method still applies: it types the supply inside the bit and claims no more than the record already holds.

Magnitude. The deciding content is a count bound, not a construction, as for the BSD half rank at most order. The method's bit is not a construction to supply; the row closes on the half that is, and the magnitude half is carried at its own grade.

The shape gate refuses one more kind of target outright: a constitutive block, a face whose deciding content is the frame itself. There nothing is supplied and nothing is closed.

3. The four inputs and the one theorem

3.1 The normal form

A row of the normal form is a type of frames, a predicate on frames, a map sending each frame to its record, and a proof that the record of every frame carries the value (`NRow`). The bit of a frame is the transfer, the record's value carried to the frame (`Transfer`). On every row the bit of a frame is exactly its value (`normal_form_once`), and every route to the value passes the bit (`every_route_passes_the_bit_once`). Both are proved once, on no axiom, and every row of the series is an instance by exact type (Islam 2026n).

3.2 What a row builds

The method asks a row for four things, and the structure `Closure` holds them. The first is a row of the normal form on the question's own objects; its field `record_has_value` is where the row's work lives, since the record must carry the value with no premise. The second is a host frame whose value is the row's sentence (`host_value`); the binding of the host to the question as the institution states it is the row's second piece of work. The third and fourth are two frames, calm and counter, that split on the value (`calm_value`, `counter_fails`); the record of each carries the value, as the record of every frame does.

3.3 What the method returns

From those four inputs the method proves, on no axiom (`the_method`):

1. the sentence is the bit: the row's sentence is exactly the transfer at the host (`sentence_is_the_bit`);
2. the record decides nothing: the record carries the value at the calm frame and at the counter frame, and the value splits between them (`record_decides_nothing`);
3. the wall: a premise shared by every frame forces nothing (`the_wall_on_the_row`);
4. the bit has no uniform answer: the transfer holds on the calm frame and fails on the counter frame (`no_uniform_answer`);
5. every route to the sentence passes the bit (`every_route_passes_the_bit`);
6. delivered twice: a formal cargo and a physical witness that each deliver the bit each give the sentence (`delivered_twice`);
7. the upgrade of the bit is the adoption of the sentence (`upgrade_is_adopting_the_sentence`).

Where the two frames share one record, the second conclusion strengthens: no predicate of the record decides the value (`record_predicate_decides_nothing`). The toy row runs the theorem: two frames, true and false, the record of each true, the host true, the counter false (`toyClosure`, `the_closure_on_its_toy_row`); its two frames share one record, so no predicate of the record decides its value (`the_toy_record_decides_nothing`). The master runs the theorem on the series' rows: on Navier–Stokes, Yang–Mills, Hodge, BSD and Poincaré at every host and on Goldbach, all seven conclusions, and on Navier–Stokes, Yang–Mills, Hodge and Goldbach the stronger form, since their calm and counter frames share one record (Islam 2026n, Section 4).

4. The ten steps, as a procedure

The series closed each row in ten steps and bound them in one theorem with nothing before its colon. The steps are the procedure; Table 1 gives, for each, what the row builds and the Lean shape of the conjunct, as the five v1.1.0 masters state it (Islam 2026a, 2026b, 2026c, 2026e, 2026f).

Steps 3, 4, 6, 9 and 10 are the method's theorem read on the row: Step 4 is the field `record_has_value`, Step 6 the two coherent frames, Step 3 the host, and Steps 9 and 10 two of the theorem's conclusions. Steps 1, 2, 5, 7 and 8 are the row's own work. Step 1 imports the root and costs nothing. Step 2 decides the row's elementary objects. Step 5 reads the row's self-recursion at the host. Step 7 is the axis, the cargo identified with the sentence. Step 8 is where the field's published mathematics enters: a cited theorem is carried as a structure field at the grade of its citation, every such field shown load-bearing by a frame on which it fails, and the value is proved to be exactly its remainder.

5. The walls, by recipe

A wall proves that a class of premises cannot supply the bit. Every wall of the series has one recipe.

Find the two frames. Build a calm frame and a counter frame that share the record and split on the value. The record is what every frame shares; if two frames share it and split, nothing read from the record decides. On the Riemann row the frames are two fold-closed worlds of one record; on P ≠ NP, the closed world and the stage world; on Navier–Stokes, a calm flow and a burst; on Yang–Mills, a calm theory and a gapless one; on Hodge, BSD and Poincaré, a calm frame and a counter frame of the row's own type.

State the class of premises as a predicate on frames. A premise every frame satisfies is keyless; one that forces the value on every frame where it holds forces. The recipe's lemma is two lines: a keyless premise holds at the counter frame, so if it forced the value it would force it there, where the value fails (`keyless_forces_nothing`). Every forcing premise is keyed, failing at the counter frame (`forcing_is_keyed`), and the value itself is the weakest key (`the_value_is_the_weakest_key`).

Specialize to the row's tempting premises. Each named route of the literature becomes a predicate and is shown keyless. The energy inequality holds on a blowup frame; the transfer trace is the same on a gapped and a gapless matrix; the sign of a centred expansion is the same at order one and order three; a chain and a cycle share their degree record. A route the recipe walls is a route that reads only what the frames share.

Lift to many rows. One principle that forces the value on every row is keyed on every row (`unified_forcing_is_keyed_on_every_row`); one keyless row breaks the union (`one_keyless_row_breaks_the_union`). On the product of the rows every pattern of values is realized (`every_pattern_is_realized`), one row's value settles no other (`one_row_settles_no_other`), and a unified principle rejects every pattern with a counter coordinate (`a_unified_principle_rejects_every_counter_pattern`).

6. The bit, delivered twice

The method never proves the bit inside the kernel. It delivers it, in two kinds, and proves that each kind yields the sentence. The formal cargo is the record's value carried to the frame: it arrives free, and the axis identifies it with the sentence. The physical witness is the sentence read on matter: the row's registration priced at the Landauer floor (Landauer 1961), the speed limit and the cascade, the record of confinement, a relaxing flow. The method's sixth conclusion is the template: whatever cargo and whatever witness deliver the bit, each gives the sentence (`delivered_twice`). Every row also proves the converse, that there is no third kind and that one counterexample to the sentence refutes both kinds at once; that is the row's refuter, and it arms the claim.

7. Grades and pinning

Four grades. Every claim carries one of four grades: on no axiom; on Lean's standard axioms alone, propext, Quot.sound and Classical.choice; at premise grade, with the premise printed as the type of a hypothesis; and cited at the grade of a published derivation, carried as a structure field. A claim is never stated above its grade, and a grade is read from the warrant, never elected (Islam 2026n, Appendix R).

Every cone pinned. Every theorem's cone is pinned by `#guard_msgs in #print axioms`, so a drifted cone fails the compile; the build is the falsifier of every grade.

One module per original. When masters are bound together, each original is a module compiled in its own environment, and none is pasted under another's imports. The synthesis measured the alternative: pasting the seven core originals under Mathlib moved 220 pinned cones and broke eight proofs, through the library's larger simp set and names the library also declares; the module tree moved none (Islam 2026n, Appendix A). Where two originals declare the same names, the shared block moves to one module and both import it, and the move is certified from the bytes.

Certificates from bytes. Every module of a bound master carries a certificate that recomputes from its published bytes: byte-identical, or the published file under one stated mechanical edit, byte for byte. A certificate that cannot be recomputed is not a certificate.

Step	What the row builds	Lean shape of the conjunct
1 · The floor	the root on no axiom, imported from the three kernels of Appendix R	`Root Unit (fun _ => 1) ∧ RA₀`
2 · The basis	the row's elementary objects, decided, beside the prime basis	decidability of the row's counts; `mulFibre 7 = [(1, 7), (7, 1)]`
3 · The domain	the host's value is the sentence, and the sentence is its own parts	`Value H.frame ↔ H.Sentence`
4 · The record	the record of every frame carries the value, the seat read off	`Value (recFrame H.frame) ∧ ∀ F d, (recFrame F).f d = seat`
5 · Self-recursion, halted	the row's self-recursive reading at the host is the sentence	`SelfRecursive row H.frame ↔ H.Sentence`
6 · Two worlds, one record	the calm and the counter frame share the record and split on value and transfer	`Value (recFrame calm) ∧ Value (recFrame counter) ∧ Value calm ∧ ¬ Value counter; Transfer row calm ∧ ¬ Transfer row counter`
7 · The axis	the formal cargo is exactly the sentence	`FormalCargo H ↔ H.Sentence`
8 · The real part	what is published or computed, proved or carried as a cited field; given it, the value exactly its remainder	`∀ S, RealPartAt …; ∀ S, Value S.F ↔ UnicornAt …`
9 · Every route passes the bit	the routes, the keyless wall, the staged records and the row's own walls	`∀ Q, (Q → H.Sentence) → Q → FormalCargo H; ∀ P, Keyless row P → ¬ Forces row P`
10 · Delivered twice	each kind of delivery yields the sentence	`(FormalCargo H → H.Sentence) ∧ (PhysicalWitness H → H.Sentence)`

Table 1 | The ten steps as a procedure. Steps 3, 4, 6, 9 and 10 are the method's theorem read on the row; Steps 1, 2, 5, 7 and 8 are the row's own work.

8. Binding to the library

The host is the row's reading of the question, and the question belongs to the institution that states it. Where the library states the question, the method binds the host to the library's statement (Mathlib Community 2020) in a separate module on Lean's standard axioms, and the bit becomes the library's statement by exact type: Mathlib's `RiemannHypothesis`, Fefferman's statement (A) written in Mathlib's calculus, the rank statement on Mathlib's group of rational points, the library's Poincaré statement (Islam 2026n). Where the library states nothing, the host is the kernel's own reading, and its fidelity to the institution's statement is the row's open flank; the falsification gate aims there.

9. The supply-side screen

Once the bit is located, the screen asks whether anything lifts it inside the kernel, and answers by proof.

The upgrade is adoption. A premise is necessary and sufficient for the sentence exactly when adopting it as an axiom proves the same propositions as adopting the sentence (`upgrade_is_adoption`); equivalence alone gives mandatory, no other proof, future-proof and the two kinds (`equivalence_gives_four`); mandatory already is necessity (`mandatory_is_necessity`). The bit's celebrated properties are the signature of its equivalence with the sentence.

The screen refuses it. The axiom generator's admission screen, written as code, refuses a candidate equivalent to its target as circular, whatever its coupling, its literature and its breadth (`the_bit_is_refused_circular`). No bit is blessed into an axiom inside the kernel.

Finite stretches. No finite stretch of verified instances forces the universal (`no_finite_stretch_forces_the_universal`); computation proves the real part height by height and never the bit.

The hunt. The screen is run as an adversarial cycle with planted controls, a literature gate before any candidate is credited with mass, and a ledger that computes the verdict. On the seven Millennium rows it ran three rounds over six registers and halted owned: no candidate firm, no road, the door on every open face owned by named programmes of the field (Islam 2026l). The screen closes the formal road; it does not close the record's.

10. The record's road

The method's last step is not in the kernel. A question stands resolved in the record when the community, having read a delivery, accepts it, and the record has done so for one Millennium question while the kernel held the statement as a premise: the Clay Institute accepted Perelman's derivation of the Poincaré bit as unconditional on 18 March 2010 (Clay Mathematics Institute 2010), and Mathlib states the theorem as `proof_wanted` to this day. The synthesis typed that derivation in the kernel as one inhabitant strictly inside the bit and proved the bit the minimal premise (Islam 2026n, Section 7). The method's closing step is to present each row to the record on the same footing: the form proved, the bit printed as a premise and pinned to the library where the library has a statement, the bit delivered twice, the refuter named. A row so presented qualifies for the standing the Poincaré bit holds.

11. The falsification gate

Each row arms at most three falsifiers, each forced by the row's own structure and aimed at what the row proved. **F-Cone:** the compile prints a cone that differs from its pin, or a certificate fails on the published bytes. **F-Row:** one counterexample to the row's sentence; it refutes both kinds of the delivery at once and withdraws the row's claim, and it moves no other row.

F-Host: a gap between the row's host and the institution's statement; at a library-bound row the falsifier must find a gap between the library's statement and the institution's, and at a row the library does not state it aims at the kernel's reading.

12. Disqualifiers

The method does not apply, and says so, in five cases. No record carries the value with no premise: then the transfer is not the whole content of the value, and the row is not in normal form. No two frames share a record and split on the value: then the bit has a uniform answer and the question is decided, by a proof or a refutation, not closed to a bit. The deciding content is a magnitude: then the bit is not a construction to supply, and the row closes only on its supply-shaped part. The face is a constitutive block: then nothing is supplied. And the host cannot be tied to the institution's statement: then the row closes its own reading and claims nothing of the institution's question.

13. Conclusion

The one-bit method takes four inputs, a row of the normal form, a host, and two frames that split on the value, and returns the closure as one theorem on no axiom: the sentence is the bit, the record decides nothing, no keyless premise forces it, the bit has no uniform answer, every route passes it, either delivery gives the sentence, and adopting the bit is adopting the sentence. Around the theorem stand the shape gate, the ten steps, the wall recipe, the two deliveries, the grade and module discipline, the binding to the library, the supply-side screen, the record's road, the falsification gate and the disqualifiers. The series ran the method on seven Millennium rows and Goldbach; the synthesis bound the eight in one master. The method closes a question to its bit and delivers the bit, and the record grants the standing.

Appendix A · The executable form

The method's file, `One_Bit_Method_v1_0_0_CC.lean`, printed in full and numbered by line: Part S, the synthesis's one normal form, verbatim; Part H, the generic half of the hunt, verbatim; Part M, the method as one theorem; and at its foot every pinned cone. It compiles on Lean 4.19.0 with no import, exit 0 and no message, in under a second; its twenty-nine theorems print *does not depend on any axioms*, each pinned by `#guard_msgs`. Its SHA-256 is `770e21c80f57df9f9f076e9b1d03decd426ded66f8281cbd0113e6966d981bcb`.

```
/-
  One_Bit_Method_v1_0_0_CC.lean · the executable form of the one-bit method
  Claude-coded (_CC), 23 October 2026. Core Lean 4.19.0: no import, no axiom declared, no sorry, no native_decide.

  The method of the Millennium series, as code. Part S is the synthesis's one normal form (the root file of the
  Millennium Grand Master, Part S.1 and S.6, verbatim). Part H is the generic half of the Perelmanification hunt
  (module Millennium.Hunt, sections H.1 to H.5 and H.8, verbatim): the seven properties as the signature of
  equivalence, the admission screen written as code, keys and walls, one principle on many rows, the product of rows,
  and finite stretches. Part M is new: the method as one theorem. A row builds four things, a row of the normal form,
  a host whose value is the row's sentence, and two coherent frames, calm and counter, splitting on the value; the
  method returns the closure: the sentence is the bit, the record decides nothing, no keyless premise forces it, the
  bit has no uniform answer, every route passes it, either delivery gives the sentence, and the upgrade of the bit is
  the adoption of the sentence. The master's root carries Part M verbatim and runs it on the series' rows.

  Every theorem below is pinned by #guard_msgs at the foot of the file; any drift fails the compile. ΔM = 0.
-/

namespace OneBitMethod

universe u

/-! ## Part S · One normal form (Millennium Grand Master, Part S.1 and S.6, verbatim) -/

/-- A ROW OF THE NORMAL FORM: frames, the value of a frame, the registration of a frame onto its record, and the
    record of every frame carrying the value. -/
structure NRow where
  Frame           : Type u
  Value           : Frame → Prop
  Rec             : Frame → Frame
  record_has_value : ∀ F, Value (Rec F)

/-- THE BIT OF A FRAME: the value carried from the frame's record to the frame. -/
def Transfer (R : NRow.{u}) (F : R.Frame) : Prop := R.Value (R.Rec F) → R.Value F

/-- THE NORMAL FORM, PROVED ONCE: on every row, the bit of a frame is exactly its value. -/
theorem normal_form_once (R : NRow.{u}) (F : R.Frame) : Transfer R F ↔ R.Value F :=
  (fun h => h (R.record_has_value F), fun h _ => h)

/-- EVERY ROUTE PASSES THE BIT, PROVED ONCE: whatever yields the value yields the bit. -/
theorem every_route_passes_the_bit_once (R : NRow.{u}) (F : R.Frame) (Q : Prop) (h : Q → R.Value F) :
    Q → Transfer R F :=
  fun q _ => h q

/-- The toy row: two frames, true and false; the record of every frame is true. -/
def toy : NRow.{0} := (Bool, fun b => b = true, fun _ => true, fun _ => rfl)

/-! ## Part H · The generic half of the hunt (module Millennium.Hunt, H.1 to H.5 and H.8, verbatim) -/

section seven
variable (Bit Sent : Prop)

/-- Necessary: every proof of the sentence yields the bit. -/
def Necessary : Prop := Sent → Bit
/-- Sufficient: the bit yields the sentence. -/
def Sufficient : Prop := Bit → Sent
/-- Mandatory: every route to the sentence is a route to the bit. -/
def Mandatory : Prop := ∀ Q : Prop, (Q → Sent) → Q → Bit
/-- No other proof: no premise yields the sentence and fails to yield the bit. -/
def NoOtherProof : Prop := ¬ ∃ Q : Prop, (Q → Sent) ∧ ¬ (Q → Bit)
/-- Future-proof: every restatement of the sentence, in any vocabulary, restates the bit. -/
def FutureProof : Prop := ∀ S : Prop, (S ↔ Sent) → (S ↔ Bit)
/-- Delivered by two kinds: whatever proves the sentence, cargo or witness, delivers the bit. -/
def TwoKinds (Cargo Witness : Prop) : Prop :=
  (Cargo → Sent) → (Witness → Sent) → (Cargo → Bit) ∧ (Witness → Bit)
/-- The standard-axiom upgrade, written out: adopting the bit as an axiom proves exactly what adopting the sentence
    proves. -/
def SameTheory : Prop := ∀ Q : Prop, (Bit → Q) ↔ (Sent → Q)

theorem necessary_and_sufficient_iff_equivalent :
    (Necessary Bit Sent ∧ Sufficient Bit Sent) ↔ (Bit ↔ Sent) :=
  (fun h => (h.2, h.1), fun e => (e.mpr, e.mp))

/-- THE UPGRADE IS ADOPTION: a premise is necessary and sufficient for the sentence exactly when adopting it as an
    axiom proves the same propositions as adopting the sentence. -/
theorem upgrade_is_adoption :
    (Necessary Bit Sent ∧ Sufficient Bit Sent) ↔ SameTheory Bit Sent :=
  (fun h _ => (fun k s => k (h.1 s), fun k b => k (h.2 b)),
   fun h => ((h Bit).mp id, (h Sent).mpr id))

/-- Equivalence alone gives four more of the seven: mandatory, no other proof, future-proof, the two kinds. -/
theorem equivalence_gives_four (e : Bit ↔ Sent) :
    Mandatory Bit Sent ∧ NoOtherProof Bit Sent ∧ FutureProof Bit Sent ∧
      ∀ C W : Prop, TwoKinds Bit Sent C W :=
  (fun _ h q => e.mpr (h q),
   fun ⟨_, h, hn⟩ => hn (fun q => e.mpr (h q)),
   fun _ h => (fun s => e.mpr (h.mp s), fun b => h.mpr (e.mp b)),
   fun _ _ hc hw => (fun c => e.mpr (hc c), fun w => e.mpr (hw w)))
```

```
88
89 /-- Mandatory already is necessity: the sentence is its own route. -/
90 theorem mandatory_is_necessity : Mandatory Bit Sent → Necessary Bit Sent
     :=
91   fun h => h Sent id
92
93 end seven
94
95 /-- The admission states of the axiom generator's screen, in its gate
     order (axiom-generator v1.3.6, Part 1.1). -/
96 inductive Admission where
97   | refusedIllformed | refusedInconsistent | refusedCircular |
       refusedInert | conditional | massCandidate
98   deriving DecidableEq, Repr
99
100 /-- What the screen reads of a candidate. -/
101 structure Reading where
102   wellformed    : Bool
103   refutedByBase : Bool
104   equivToTarget : Bool
105   couplingProof : Bool
106   inLiterature  : Bool
107   breadth       : Nat
108
109 /-- The screen gate by gate, S3, A1, A2, A3, A4, A5, as `axf` in
     axiom_generator.py v1.3.6. -/
110 def screen (r : Reading) : Admission :=
111   match r.wellformed, r.refutedByBase, r.equivToTarget, r.couplingProof,
       r.inLiterature, r.breadth with
112   | false, _, _, _, _, _ => .refusedIllformed
113   | true, true, _, _, _, _ => .refusedInconsistent
114   | true, false, true, _, _, _ => .refusedCircular
115   | true, false, false, false, _, _ => .refusedInert
116   | true, false, false, true, true, _ => .conditional
117   | true, false, false, true, false, 0 => .conditional
118   | true, false, false, true, false, _ + 1 => .massCandidate
119
120 /-- The reading of the unified bit at the screen: a sentence of the base
     language, refuted by nothing known, and
121     equivalent to its target by its own necessity and sufficiency (H.1,
       H.7). Coupling, literature and breadth are left
122     free: the verdict does not read them. -/
123 def bitReading (c l : Bool) (b : Nat) : Reading := ⟨true, false, true, c,
     l, b⟩
124
125 /-- THE UPGRADE IS REFUSED CIRCULAR AT A2, whatever the coupling, the
     literature and the breadth. -/
126 theorem the_bit_is_refused_circular (c l : Bool) (b : Nat) :
127     screen (bitReading c l b) = Admission.refusedCircular := rfl
128
129 /-- A row as the hunt reads it: frames, the value of a frame, and the two
     coherent frames every row kernel builds,
130     one carrying the value and one failing it. -/
131 structure KRow where
132   Frame         : Type u
133   Value         : Frame → Prop
134   calm          : Frame
135   counter       : Frame
136   calm_value    : Value calm
137   counter_fails : ¬ Value counter
138
139 section rowlaws
140 variable (K : KRow.{u})
141
142 /-- A premise shared by every frame. -/
143 def Keyless (P : K.Frame → Prop) : Prop := ∀ F, P F
144 /-- A premise that forces the value on every frame where it holds. -/
145 def Forces (P : K.Frame → Prop) : Prop := ∀ F, P F → K.Value F
146 /-- A keyed premise: it fails at the counter frame. -/
147 def Keyed (P : K.Frame → Prop) : Prop := ¬ P K.counter
148
149 theorem keyless_forces_nothing (P : K.Frame → Prop) (hP : Keyless K P) :
     ¬ Forces K P :=
150   fun h => K.counter_fails (h K.counter (hP K.counter))
151
152 /-- EVERY FORCING PREMISE IS KEYED: what forces the value fails where the
     value fails. -/
153 theorem forcing_is_keyed (P : K.Frame → Prop) (h : Forces K P) : Keyed K
     P :=
154   fun hc => K.counter_fails (h K.counter hc)
155
156 /-- THE VALUE IS THE WEAKEST KEY: it forces itself, and every forcing
     premise implies it. -/
157 theorem the_value_is_the_weakest_key :
158     Forces K K.Value ∧ ∀ P : K.Frame → Prop, Forces K P → ∀ F, P F →
         K.Value F :=
159   ⟨fun _ h => h, fun _ h => h⟩
160
161 end rowlaws
162
163 section family
164 variable {ι : Type} (K : ι → KRow.{u})
165
166 /-- ONE PRINCIPLE FORCING EVERY ROW IS KEYED ON EVERY ROW. -/
167 theorem unified_forcing_is_keyed_on_every_row (U : ∀ i, (K i).Frame →
     Prop)
168     (h : ∀ i, Forces (K i) (U i)) : ∀ i, Keyed (K i) (U i) :=
169   fun i => forcing_is_keyed (K i) (U i) (h i)
170
171 /-- One keyless row breaks the union. -/
172 theorem one_keyless_row_breaks_the_union (U : ∀ i, (K i).Frame → Prop) (j
     : ι)
173     (hj : Keyless (K j) (U j)) : ¬ ∀ i, Forces (K i) (U i) :=
174   fun h => keyless_forces_nothing (K j) (U j) hj (h j)
175
176 /-- A frame of the product: one frame on every row. -/
177 def ProdFrame : Type u := ∀ i, (K i).Frame
178
179 /-- The pattern frame: the calm frame where the pattern reads true, the
     counter frame where it reads false. -/
180 def pick (S : ι → Bool) : ProdFrame K := fun i => cond (S i) (K i).calm
     (K i).counter
181
182 theorem pick_realizes (S : ι → Bool) (i : ι) : (K i).Value (pick K S i) ↔
     S i = true := by
183   show (K i).Value (cond (S i) (K i).calm (K i).counter) ↔ S i = true
184   cases S i with
185   | true => exact ⟨fun _ => rfl, fun _ => (K i).calm_value⟩
186   | false => exact ⟨fun hv => absurd hv (K i).counter_fails, fun hf =>
       Bool.noConfusion hf⟩
187
188 /-- THE VALUES ARE INDEPENDENT: every pattern of values over the rows is
     realized by a frame of the product. -/
189 theorem every_pattern_is_realized (S : ι → Bool) :
190     ∃ F : ProdFrame K, ∀ i, (K i).Value (F i) ↔ S i = true :=
191   ⟨pick K S, pick_realizes K S⟩
192
193 /-- ONE ROW SETTLES NO OTHER: a frame of the product carries the value on
     row p and fails it on row i. -/
194 theorem one_row_settles_no_other [DecidableEq ι] (p i : ι) (hpi : p ≠ i) :
195     ∃ F : ProdFrame K, (K p).Value (F p) ∧ ¬ (K i).Value (F i) :=
196   ⟨pick K (fun j => decide (j = p)),
197    ⟨(pick_realizes K _ p).mpr (decide_eq_true rfl),
198     fun hv => hpi (of_decide_eq_true ((pick_realizes K _ i).mp hv)).symm⟩⟩
199
200 /-- A UNIFIED PRINCIPLE REJECTS EVERY COUNTER PATTERN: one premise on the
     product that forces every row's value is
201     false at every pattern frame with a counter coordinate, so it carries
         a key on every coordinate. -/
202 theorem a_unified_principle_rejects_every_counter_pattern (U : ProdFrame
     K → Prop)
203     (hU : ∀ F, U F → ∀ i, (K i).Value (F i)) (S : ι → Bool) (j : ι) (hj :
         S j = false) :
204     ¬ U (pick K S) :=
205   fun hu => Bool.noConfusion (((pick_realizes K S j).mp (hU _ hu
       j)).symm.trans hj)
206
207 /-- A principle shared by every product frame forces no row. -/
208 theorem a_keyless_union_forces_no_row (U : ProdFrame K → Prop) (hU : ∀ F,
     U F) (j : ι) :
209     ¬ ∀ F, U F → (K j).Value (F j) :=
210   fun h => (K j).counter_fails (h (pick K (fun _ => false)) (hU _))
211
212 end family
213
214 /-- NO FINITE STRETCH FORCES THE UNIVERSAL: for every height there is a
     predicate true at every instance up to that
215     height and false one step beyond. -/
216 theorem no_finite_stretch_forces_the_universal (N : Nat) :
217     ∃ c : Nat → Bool, (∀ n, n ≤ N → c n = true) ∧ c (N + 1) = false :=
218   ⟨fun n => decide (n ≤ N), fun _ h => decide_eq_true h, decide_eq_false
       (Nat.not_succ_le_self N)⟩
219
220 /-! ## Part M · The method, as one theorem (new) -/
221
222 /-- WHAT A ROW BUILDS: a row of the normal form; a host frame whose value
     is the row's sentence; and two coherent
223     frames, a calm frame carrying the value and a counter frame failing
         it, the record of each carrying the value.
224     These are the method's four inputs; everything else is returned. -/
225 structure Closure where
226   R             : NRow.{u}
227   Sentence      : Prop
228   host          : R.Frame
229   host_value    : R.Value host ↔ Sentence
230   calm          : R.Frame
231   counter       : R.Frame
232   calm_value    : R.Value calm
233   counter_fails : ¬ R.Value counter
234
235 namespace Closure
236 variable (C : Closure.{u})
237
238 /-- The closure read as a row of the hunt: its two frames are the calm
     and the counter frame. -/
239 def toK : KRow.{u} := ⟨C.R.Frame, C.R.Value, C.calm, C.counter,
     C.calm_value, C.counter_fails⟩
240
241 /-- THE SENTENCE IS THE BIT: at the host, the row's sentence is exactly
     the transfer. -/
242 theorem sentence_is_the_bit : C.Sentence ↔ Transfer C.R C.host :=
243   C.host_value.symm.trans (normal_form_once C.R C.host).symm
244
245 /-- THE RECORD DECIDES NOTHING: the record carries the value at the calm
     frame and at the counter frame, and the
246     value splits between them. -/
247 theorem record_decides_nothing :
248     C.R.Value (C.R.Rec C.calm) ∧ C.R.Value (C.R.Rec C.counter) ∧
         C.R.Value C.calm ∧ ¬ C.R.Value C.counter :=
249   ⟨C.R.record_has_value C.calm, C.R.record_has_value C.counter,
       C.calm_value, C.counter_fails⟩
250
251 /-- ONE RECORD, NO DECIDING PREDICATE: where the two frames share one
     record, no predicate of the record decides the
252     value. -/
253 theorem record_predicate_decides_nothing (h : C.R.Rec C.calm = C.R.Rec
     C.counter) :
254     ¬ ∃ g : C.R.Frame → Prop, ∀ F, C.R.Value F ↔ g (C.R.Rec F) :=
255   fun ⟨_, hg⟩ => C.counter_fails ((hg C.counter).mpr (h ▸ (hg C.calm).mp
       C.calm_value))
256
257 /-- THE WALL: a premise shared by every frame forces nothing. -/
```

```
theorem the_wall_on_the_row (P : C.R.Frame → Prop) (hP : ∀ F, P F) : ¬ ∀ F, P F → C.R.Value F :=
  keyless_forces_nothing C.toK P hP

/-- THE BIT HAS NO UNIFORM ANSWER: the transfer holds on the calm frame and fails on the counter frame. -/
theorem no_uniform_answer : ¬ (∀ F, Transfer C.R F) ∧ ¬ (∀ F, ¬ Transfer C.R F) :=
  (fun h => C.counter_fails (h C.counter (C.R.record_has_value C.counter)),
   fun h => h C.calm (fun _ => C.calm_value))

/-- EVERY ROUTE PASSES THE BIT: whatever premise yields the sentence yields the transfer at the host. -/
theorem every_route_passes_the_bit (Q : Prop) (h : Q → C.Sentence) : Q → Transfer C.R C.host :=
  fun q => C.sentence_is_the_bit.mp (h q)

/-- DELIVERED TWICE: a formal cargo and a physical witness, each delivering the bit, each give the sentence. -/
theorem delivered_twice (Cargo Witness : Prop) (hc : Cargo → Transfer C.R C.host)
    (hw : Witness → Transfer C.R C.host) : (Cargo → C.Sentence) ∧ (Witness → C.Sentence) :=
  (fun c => C.sentence_is_the_bit.mpr (hc c), fun w => C.sentence_is_the_bit.mpr (hw w))

/-- THE UPGRADE OF THE BIT IS THE ADOPTION OF THE SENTENCE. -/
theorem upgrade_is_adopting_the_sentence : SameTheory (Transfer C.R C.host) C.Sentence :=
  (upgrade_is_adoption _ _).mp ((necessary_and_sufficient_iff_equivalent _ _).mpr C.sentence_is_the_bit.symm)

/-- THE METHOD, as one theorem with nothing before its colon beyond the four inputs: the sentence is the bit; the
    record decides nothing; no keyless premise forces the value; the bit has no uniform answer; every route passes
    the bit; either delivery gives the sentence; and the upgrade of the bit is the adoption of the sentence. -/
theorem the_method :
    (C.Sentence ↔ Transfer C.R C.host) ∧
    (C.R.Value (C.R.Rec C.calm) ∧ C.R.Value (C.R.Rec C.counter) ∧ C.R.Value C.calm ∧ ¬ C.R.Value C.counter) ∧
    (∀ P : C.R.Frame → Prop, (∀ F, P F) → ¬ ∀ F, P F → C.R.Value F) ∧
    (¬ (∀ F, Transfer C.R F) ∧ ¬ (∀ F, ¬ Transfer C.R F)) ∧
    (∀ Q : Prop, (Q → C.Sentence) → Q → Transfer C.R C.host) ∧
    (∀ Cargo Witness : Prop, (Cargo → Transfer C.R C.host) → (Witness → Transfer C.R C.host) →
      (Cargo → C.Sentence) ∧ (Witness → C.Sentence)) ∧
    SameTheory (Transfer C.R C.host) C.Sentence :=
  (C.sentence_is_the_bit, C.record_decides_nothing, C.the_wall_on_the_row, C.no_uniform_answer,
   C.every_route_passes_the_bit, C.delivered_twice, C.upgrade_is_adopting_the_sentence)

end Closure

/-- THE METHOD ON ITS TOY ROW: the toy row with host true, sentence `true = true`, calm true and counter false. -/
def toyClosure : Closure.{0} where
  R := toy
  Sentence := true = true
  host := true
  host_value := Iff.rfl
  calm := true
  counter := false
  calm_value := rfl
  counter_fails := Bool.noConfusion

theorem the_closure_on_its_toy_row : type_of% (Closure.the_method toyClosure) := Closure.the_method toyClosure

/-- On the toy row the two frames share one record, so no predicate of the record decides the value. -/
theorem the_toy_record_decides_nothing :
    ¬ ∃ g : toyClosure.R.Frame → Prop, ∀ F, toyClosure.R.Value F ↔ g (toyClosure.R.Rec F) :=
  Closure.record_predicate_decides_nothing toyClosure rfl

end OneBitMethod

/-! ## Cones, pinned -/
/-- info: 'OneBitMethod.normal_form_once' does not depend on any axioms -/
#guard_msgs in #print axioms OneBitMethod.normal_form_once
/-- info: 'OneBitMethod.every_route_passes_the_bit_once' does not depend on any axioms -/
#guard_msgs in #print axioms OneBitMethod.every_route_passes_the_bit_once
/-- info: 'OneBitMethod.necessary_and_sufficient_iff_equivalent' does not depend on any axioms -/
#guard_msgs in #print axioms OneBitMethod.necessary_and_sufficient_iff_equivalent
/-- info: 'OneBitMethod.upgrade_is_adoption' does not depend on any axioms -/
#guard_msgs in #print axioms OneBitMethod.upgrade_is_adoption
/-- info: 'OneBitMethod.equivalence_gives_four' does not depend on any axioms -/
#guard_msgs in #print axioms OneBitMethod.equivalence_gives_four
/-- info: 'OneBitMethod.mandatory_is_necessity' does not depend on any axioms -/
#guard_msgs in #print axioms OneBitMethod.mandatory_is_necessity
/-- info: 'OneBitMethod.the_bit_is_refused_circular' does not depend on any axioms -/
#guard_msgs in #print axioms OneBitMethod.the_bit_is_refused_circular
/-- info: 'OneBitMethod.keyless_forces_nothing' does not depend on any axioms -/
#guard_msgs in #print axioms OneBitMethod.keyless_forces_nothing
/-- info: 'OneBitMethod.forcing_is_keyed' does not depend on any axioms -/
#guard_msgs in #print axioms OneBitMethod.forcing_is_keyed
/-- info: 'OneBitMethod.the_value_is_the_weakest_key' does not depend on any axioms -/
#guard_msgs in #print axioms OneBitMethod.the_value_is_the_weakest_key
/-- info: 'OneBitMethod.unified_forcing_is_keyed_on_every_row' does not depend on any axioms -/
#guard_msgs in #print axioms OneBitMethod.unified_forcing_is_keyed_on_every_row
/-- info: 'OneBitMethod.one_keyless_row_breaks_the_union' does not depend on any axioms -/
#guard_msgs in #print axioms OneBitMethod.one_keyless_row_breaks_the_union
/-- info: 'OneBitMethod.pick_realizes' does not depend on any axioms -/
#guard_msgs in #print axioms OneBitMethod.pick_realizes
/-- info: 'OneBitMethod.every_pattern_is_realized' does not depend on any axioms -/
#guard_msgs in #print axioms OneBitMethod.every_pattern_is_realized
/-- info: 'OneBitMethod.one_row_settles_no_other' does not depend on any axioms -/
#guard_msgs in #print axioms OneBitMethod.one_row_settles_no_other
/-- info: 'OneBitMethod.a_unified_principle_rejects_every_counter_pattern' does not depend on any axioms -/
#guard_msgs in #print axioms OneBitMethod.a_unified_principle_rejects_every_counter_pattern
/-- info: 'OneBitMethod.a_keyless_union_forces_no_row' does not depend on any axioms -/
#guard_msgs in #print axioms OneBitMethod.a_keyless_union_forces_no_row
/-- info: 'OneBitMethod.no_finite_stretch_forces_the_universal' does not depend on any axioms -/
#guard_msgs in #print axioms OneBitMethod.no_finite_stretch_forces_the_universal
/-- info: 'OneBitMethod.Closure.sentence_is_the_bit' does not depend on any axioms -/
#guard_msgs in #print axioms OneBitMethod.Closure.sentence_is_the_bit
/-- info: 'OneBitMethod.Closure.record_decides_nothing' does not depend on any axioms -/
#guard_msgs in #print axioms OneBitMethod.Closure.record_decides_nothing
/-- info: 'OneBitMethod.Closure.record_predicate_decides_nothing' does not depend on any axioms -/
#guard_msgs in #print axioms OneBitMethod.Closure.record_predicate_decides_nothing
/-- info: 'OneBitMethod.Closure.the_wall_on_the_row' does not depend on any axioms -/
#guard_msgs in #print axioms OneBitMethod.Closure.the_wall_on_the_row
/-- info: 'OneBitMethod.Closure.no_uniform_answer' does not depend on any axioms -/
#guard_msgs in #print axioms OneBitMethod.Closure.no_uniform_answer
/-- info: 'OneBitMethod.Closure.every_route_passes_the_bit' does not depend on any axioms -/
#guard_msgs in #print axioms OneBitMethod.Closure.every_route_passes_the_bit
/-- info: 'OneBitMethod.Closure.delivered_twice' does not depend on any axioms -/
#guard_msgs in #print axioms OneBitMethod.Closure.delivered_twice
/-- info: 'OneBitMethod.Closure.upgrade_is_adopting_the_sentence' does not depend on any axioms -/
#guard_msgs in #print axioms OneBitMethod.Closure.upgrade_is_adopting_the_sentence
/-- info: 'OneBitMethod.Closure.the_method' does not depend on any axioms -/
#guard_msgs in #print axioms OneBitMethod.Closure.the_method
/-- info: 'OneBitMethod.the_closure_on_its_toy_row' does not depend on any axioms -/
#guard_msgs in #print axioms OneBitMethod.the_closure_on_its_toy_row
/-- info: 'OneBitMethod.the_toy_record_decides_nothing' does not depend on any axioms -/
#guard_msgs in #print axioms OneBitMethod.the_toy_record_decides_nothing
```

Author's Provenance and Method Disclosure

This volume was developed under Trisduction, a verification and organizing discipline that adds the cited results no warrant; the method and its executable batteries are stated in full in the master reference (Islam 2026j). *Author and method.* Mohammad F. Islam, PhD, independent researcher, sole author and sole authority for every claim and every error. The method was directed by the author and drafted with Claude (Anthropic) as scribe: the model wrote and compiled the executable form, checked its Parts S and H verbatim against the master's root and the hunt module, and checked the citations; it holds no authorship. *Audit.* This edition has not yet been prosecuted by the external substrates, and no audit result is claimed.

References

Clay Mathematics Institute. 2010. Prize for resolution of the Poincaré conjecture awarded to Dr. Grigoriy Perelman. Press release, 18 March 2010.

de Moura, L. and S. Ullrich. 2021. The Lean 4 theorem prover and programming language. *Automated Deduction, CADE 28*, 625–635.

Islam, M. F. 2026a. Navier–Stokes global regularity: unconditional proof of the form by theorem, the value by the act. Edition 1.1.0. Zenodo. doi:10.5281/zenodo.22670344.

Islam, M. F. 2026b. The Hodge conjecture: unconditional proof of the form by theorem, the value by the act. Edition 1.1.0. Zenodo. doi:10.5281/zenodo.22701510.

Islam, M. F. 2026c. Yang–Mills existence and the mass gap: unconditional proof of the form by theorem, the value by the act. Edition 1.1.0. Zenodo. doi:10.5281/zenodo.23162231.

Islam, M. F. 2026d. The Goldbach conjecture: unconditional proof of the form by theorem, the value by the act. Edition 1.1.0. Zenodo. doi:10.5281/zenodo.23162238.

Islam, M. F. 2026e. The Poincaré conjecture: unconditional proof of the form by theorem, the value by the act. Edition 1.1.0. Zenodo. doi:10.5281/zenodo.23162240.

Islam, M. F. 2026f. Birch and Swinnerton-Dyer: unconditional proof of the form by theorem, the value by the act. Edition 1.1.0. Zenodo. doi:10.5281/zenodo.23164992.

Islam, M. F. 2026g. An unconditional proof of the real part of the Riemann Hypothesis, the probable remainder closed to symmetry and computation at theorem grade, and the whole proved from one bit supplied by the act of computation or by physics itself. Edition 1.3.0. Zenodo. doi:10.5281/zenodo.23440382.

Islam, M. F. 2026h. P $\neq$ NP: unconditional proof of the form by theorem, the value by the act. Edition 1.13.0. Zenodo. doi:10.5281/zenodo.23528398.

Islam, M. F. 2026j. TRISDUCTION: A linguistically, topologically, and mathematically sealed verification architecture. Zenodo, version 4, 19 June 2026. doi:10.5281/zenodo.20757507. Master reference, continuously updated at the same location.

Islam, M. F. 2026l. The Perelmanification hunt: the one-bit posit, axiom cycle pmf1, Pre-Forge digest and ledger. Edition 1.0.0_CC, 23 October 2026. Supplement S1 to the synthesis.

Islam, M. F. 2026n. The Millennium Problems: unconditional proof of the form by theorem, the value by the act. Edition 1.0.0_CC, 23 October 2026, with its master Millennium_Grand_Master_v1_0_0_CC.

Landauer, R. 1961. Irreversibility and heat generation in the computing process. *IBM Journal of Research and Development* 5: 183–191.

Mathlib Community. 2020. The Lean mathematical library. In *Proceedings of the 9th ACM SIGPLAN International Conference on Certified Programs and Proofs*, 367–381.

End of manuscript